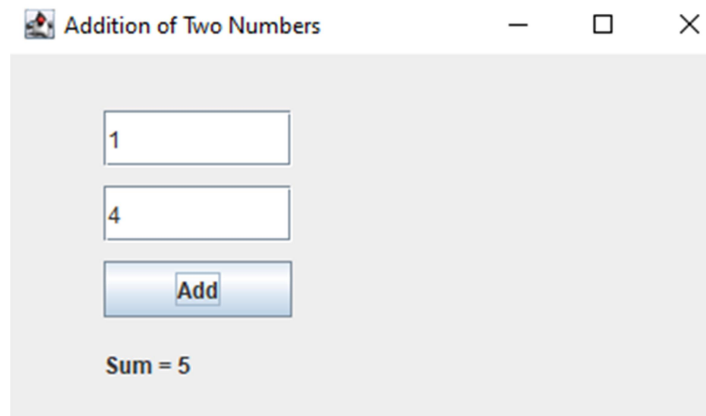


```

1 } public void actionPerformed(ActionEvent e) {
    try { int num1 = Integer.parseInt(num1field.getText());
        int num2 = Integer.parseInt(num2field.getText());
        int sum = num1 + num2;
        resultLabel.setText("Sum = " + sum);
    } catch (NumberFormatException ex) {
        resultLabel.setText("Please enter valid integers");
    }
}
2 public static void main(String[] args) {
    new SumCalculator();
}
}

```

Observation



Conclusion

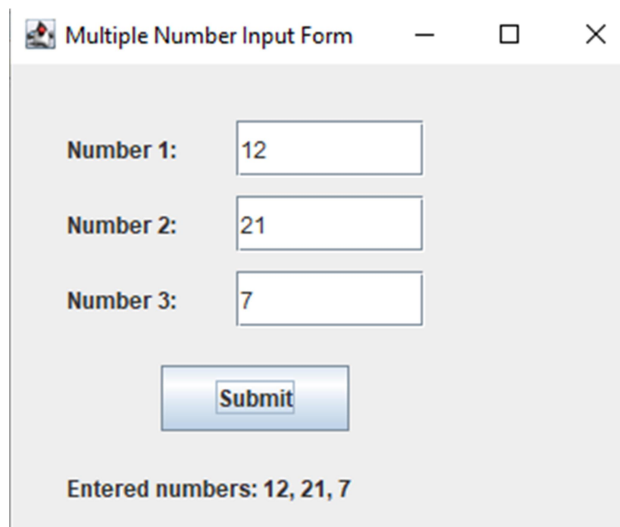
The lab demonstrated how to use Java Swing to create a simple GUI application that takes two numbers as input and calculates their sum. This exercise helped understand GUI components, event handling, and basic user input validation in Java.

```

        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
    public void actionPerformed (ActionEvent e) {
        try {
            int a = Integer.parseInt(n1.getText());
            int b = Integer.parseInt(n2.getText());
            int c = Integer.parseInt(n3.getText());
            result.setText ("Entered: " + a + " " + b + " " + c);
        } catch (Exception ex) {
            result.setText ("Enter valid integers!"),
        }
    }
    public static void main (String[] args) {
        new MultipleInputForm();
    }
}

```

Observation



Multiple Number Input Form

Number 1: 12

Number 2: 21

Number 3: 7

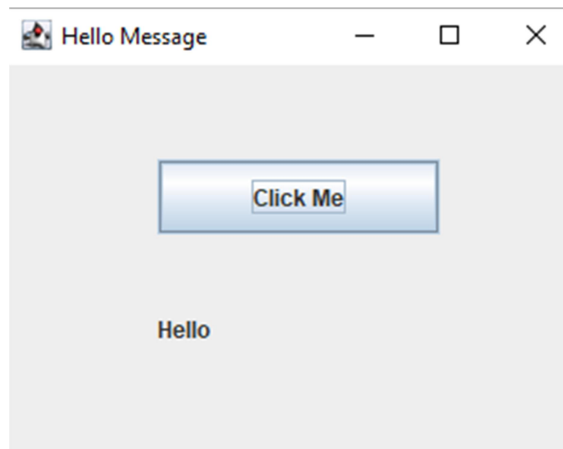
Submit

Entered numbers: 12, 21, 7

Conclusion

This lab demonstrated the creation of a form in Java Swing to accept multiple number inputs from the user. It helped understand GUI design, form handling, input validation, and event-driven programming using Swing.

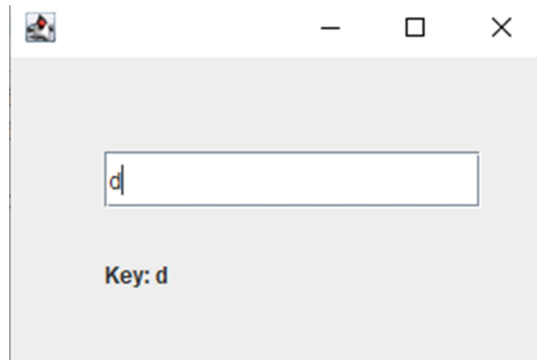
Observation



Conclusion

This experiment demonstrated how to handle button click events in Swing using ActionListener. The program successfully displays a "Hello" message when the user clicks the button, illustrating basic event-driven GUI programming.

Observation



Conclusion

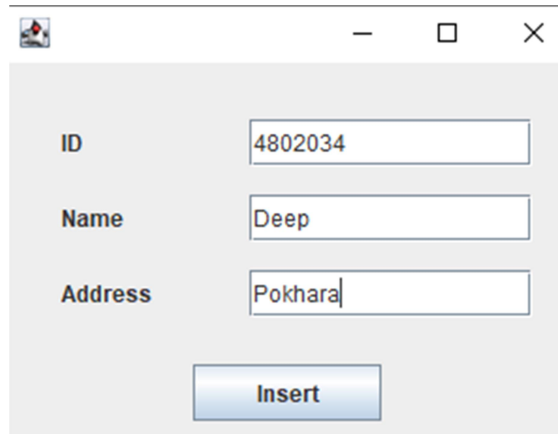
This experiment demonstrated how to handle keyboard events in Java Swing using the `KeyAdapter` class. It allowed detecting key presses in real-time and displayed the pressed key on the GUI, reinforcing understanding of event-driven programming concepts.

```

    public static void main (String[] args){
        new StudentInsert();
    }
}

```

Observation



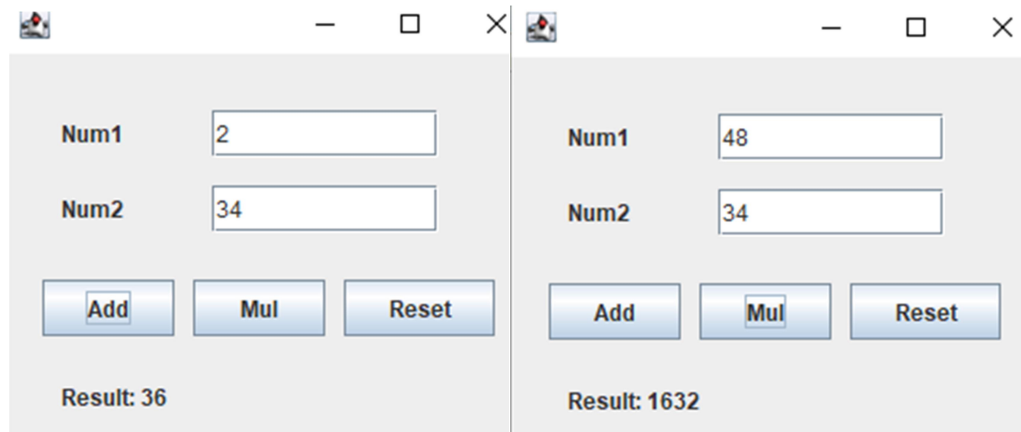
A screenshot of a Java Swing window titled "Student Insert". The window has a standard title bar with minimize, maximize, and close buttons. Inside the window, there are three text input fields labeled "ID", "Name", and "Address". The "ID" field contains the text "4802034", the "Name" field contains "Deep", and the "Address" field contains "Pokhara". Below these fields is a blue button labeled "Insert".

					id	name	address
☐	Edit	Copy	Delete		4802034	Deep	Pokhara
↑	☐ Check all	With selected:		Edit	Copy		

Conclusion

This lab successfully demonstrates how to collect student data using a Swing form and insert it into a MySQL database using JDBC. It provides practical understanding of GUI development, database connectivity, SQL operations, and event handling in Java.

Observation



Conclusion

This lab demonstrated how to build a simple calculator using Java Swing. It covered GUI form creation, button event handling, arithmetic operations, and resetting input fields, strengthening understanding of interactive application development.